

Teoretická informatika (TIN) – 2025/2026

Úkol 2

(max. zisk 8 bodů)

1. Klasifikujte příslušnost jazyků L_1 a L_2 do tříd REC (rekurzivní) a RE (rekurzivně vyčíslitelné) a dokažte (ne-)příslušnost do těchto tříd.

- $L_1 = \{\langle M \rangle \# \langle G \rangle \mid M \text{ je TS a } G \text{ je gramatika typu 2, t.ž. } \exists w \in \{0, 1\}^* : w \in L(M) \wedge w \notin L(G)\}$
- $L_2 = \{\langle M \rangle \# \langle G \rangle \mid M \text{ je TS a } G \text{ je gramatika typu 2, t.ž. } \exists w \in \{0, 1\}^* : w \notin L(M) \wedge w \in L(G)\}$

1.5 body

2. Pro neorientovaný graf definujme problém obarvení grafu k -barvami s j -chybami následovně: Nechť $G = (V, E)$ je neorientovaný graf, $k \in \mathbb{N}, k > 0$ počet barev, $j \in \mathbb{N}, j > 0$ počet povolených chyb. Graf G je obarvitelný k barvami s nejvýše j chybami pokud:

- (a) existuje funkce $c : V \rightarrow \langle 1, k \rangle$ (každý vrchol grafu obarvíme nějakou barvou)
- (b) $\exists E' \subseteq E : |E| - |E'| \leq j$ (existuje podmnožina hran původního grafu, která vznikla odstraněním maximálně j hran z původního grafu)
- (c) $\forall \{a, b\} \in E' : c(a) \neq c(b)$ (každá hrana z E' propojuje vrcholy obarvené jinou barvou)

Dokažte, že problém obarvení s chybami je NP-úplný.

(Pozn: Pomůže Vám NP-úplnost některého z problémů uvedených zde:

https://en.wikipedia.org/wiki/NP-completeness#NP-complete_problems

v odstavci „NP-complete problems“.)

1.5 body

3. Mějme teorii T se signaturou $\langle \{Cat_{/0}, Fish_{/0}, Trex_{/0}\}, \{eats_{/2}, =_{/2}\} \rangle$ (= je standardní rovnost) se speciálními axiomy

$$\begin{aligned} \forall x (x = Fish \vee x = Cat \vee x = Trex) \\ \forall x \forall y (eats(x, y) \leftrightarrow \neg eats(y, x)) \\ \forall x (x = Trex \vee eats(Trex, x)) \end{aligned}$$

- i) Rozhodněte a stručně zdůvodněte, zda T je (a) bezesporná, (b) úplná a (c) rozhodnutelná (tj. množina důsledků T je rozhodnutelná).
- ii) Rozhodněte a stručně zdůvodněte, zda může být Peanova aritmetika T_{PA} redukovatelná na T (tj. zda může existovat vyčíslitelná redukce f , která každou větu φ Peanovy aritmetiky převede na větu $f(\varphi)$ jazyka teorie T takovou, že $T_{PA} \models \varphi \iff T \models f(\varphi)$).

1 bodů

4. Uvažujeme pole celých čísel proměnné délky, jehož datová struktura obsahuje proměnné (i) *arr*—ukazatel na začátek pole, (ii) *size*—velikost alokovaného prostoru, (iii) *used*—počet využitých položek od začátku. Pro práci s polem máme následující funkce:

- *array_init*: alokuje pole velikosti 10 čísel (*malloc*) a nastaví *size* a *used*.
- *insert*: v případě, že je volné místo, vloží prvek *val* na konec pole a zvýší počet obsazených pozic o 1. V případě, že volné místo není (*size = used*), tak alokuje nové pole čísel *tmp* o velikosti *size * 2*, zkopíruje obsah původního pole do nového, dealokuje staré pole (*free*) a ukazatel na nové pole uloží do proměnné *arr*. Poté vloží prvek *val* do zvětšeného pole a obsah pole nechá seřadit funkcí *sort*.
- *sort*: seřadí pole pomocí modifikovaného bubble sortu.

(i) Analyzujte a zdůvodněte složitost jednoho volání *insert* v nejhorším případě.

(ii) S využitím amortizované složitosti analyzujte a zdůvodněte složitost posloupnosti *n* operací *insert* aplikované na pole inicializované funkcí *array_init*.

Poznámky:

- Předpokládejme uniformní cenové kritérium, kde každý řádek má cenu 1 (s výjimkou volání funkce *sort*).
- Funkce *swap(a, b)* prohodí položky hodnoty proměnných *a, b*.

```

1 Struct array
2   | int arr[];
3   | int size, used;
4 Function array_init(array a)
5   | a.arr := malloc(10);
6   | a.size := 10;
7   | a.used := 0;
8 Function insert(int val, array a)
9   | resize := false;
10  | if a.used = a.size then
11    | resize := true;
12    | tmp[] := malloc(a.size * 2);
13    | for (i := 0; i < size; i++) do
14      | tmp[i] = a.arr[i];
15    | free(a.arr);
16    | a.arr := tmp;
17    | a.size := a.size * 2;
18  | a.arr[a.used] := val;
19  | a.used := a.used + 1;
20  | if resize then
21    | sort(a);
22 Function sort(array a)
23  | sorted := false;
24  | while ¬sorted do
25    | sorted := true;
26    | for (i := a.used - 1; i > 0; i = i - 1) do
27      | if a.arr[i] < a.arr[i - 1] then
28        | swap(a.arr[i], a.arr[i - 1]);
29    | sorted := false;

```

2 body

5. Nechť $G = (V, E)$ je orientovaný graf. Definujeme sled jako

$sled(v_1, v_n) \Leftrightarrow \exists$ posloupnost $v_1 e_1 v_2 e_2 \dots e_{n-1} v_n : \forall 0 < i < n : e_i = (v_i, v_{i+1})$ a $e_i \in E$. Uzly a hrany se mohou v posloupnosti opakovat.

Pro graf $G = (V, E)$, uzly $v_s, v_f \in V$ a hranu $e \in E$ uvažujme problém existence sledu $sled(v_s, v_f)$, který obsahuje hranu e .

Uvažujme funkci *check*, která prohledá všechny sledy do délky $2 * |V|$ a zjistí, zda takovýto sled existuje.

- Analyzujte časovou složitost funkce *check* v nejlepším případě
- Analyzujte časovou složitost funkce *check* v nejhorším případě.
- Navrhněte funkci *check_opt*, která bude dávat stejný výsledek jako funkce *check*, ale bude mít lepší asymptotickou složitost v nejhorším případě.
- Analyzujte časovou složitost funkce *check_opt* v nejhorším případě.

```

1 Function check  $((V, E), v_s, v_f, e)$ 
2    $s := \text{empty\_stack}();$ 
3    $s.\text{push}((v_s, \text{false}, 0));$ 
4   while stack_notempty( $s$ ) do
5      $v, \text{visited}, \text{depth} := s.\text{pop}();$ 
6     foreach  $w \in \text{succ}(v)$  do
7        $\text{visited}_{\text{new}} := \text{visited};$ 
8       if  $(v, w) = e$  then
9          $\text{visited}_{\text{new}} := \text{true};$ 
10      if  $\text{visited}_{\text{new}} \wedge w = v_f$  then
11        return true;
12      if  $\text{depth} < 2|V|$  then
13         $s.\text{push}((w, \text{visited}_{\text{new}}, \text{depth} + 1))$ 
14  return false;
```

Poznámky:

- Předpokládejme uniformní cenové kritérium, kde každý řádek má cenu 1.
- Funkce *succ*(v) vrací všechny následníky vrcholu v v $O(1)$.
- Analýzu proveďte vzhledem k počtu vrcholů $|V|$. V případě *check_opt* můžete analyzovat vyhledem k počtu vrcholů $|V|$ a hran $|E|$.
- Do zásobníku vkládáme trojice $(v, \text{visited}, \text{depth})$, kde v je vrchol, *visited* udává, jestli jsme prošli hranou e a *depth* aktuální délku zanoření (délku sledu).

2 body